



IMPORTING LIBRARIES

```
In [1]: import numpy as npimport pandas as pd
```

```
In [2]: import matplotlib.pyplot as pltimport seaborn as sns%matplotlib inlineimport w
```

READING THE DATA

```
In [3]: superster_sales = pd.read_excel('US Superstore data.xls')superster_sales.saml
```

```
Out[3]:
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name
--	--------	----------	------------	-----------	-----------	-------------	---------------

3167	3168	CA-2016-146682	2016-10-29	2016-10-31	First Class	KW-16435	Katri Willm
-------------	------	----------------	------------	------------	-------------	----------	-------------

524	525	US-2016-139486	2016-05-21	2016-05-23	First Class	LH-17155	Logi Haushalt
------------	-----	----------------	------------	------------	-------------	----------	---------------

7159	7160	CA-2015-145835	2015-05-13	2015-05-18	Second Class	BF-11170	Ben Ferr
-------------	------	----------------	------------	------------	--------------	----------	----------

7500	7501	CA-2017-121048	2017-07-14	2017-07-18	Standard Class	TC-21295	Tol Carlis
-------------	------	----------------	------------	------------	----------------	----------	------------

2152	2153	CA-2015-123505	2015-11-22	2015-11-26	Standard Class	AR-10540	Ani Reit
-------------	------	----------------	------------	------------	----------------	----------	----------

5 rows × 21 columns

EXPLORATORY DATA

```
In [4]: df = superster_sales.copy()
```

```
In [5]: df.shape
```

```
Out[5]: (9994, 21)
```

```
In [6]: df.columns
```

```
Out[6]: Index(['Row ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mode', 'Customer ID', 'Customer Name', 'Segment', 'Country', 'City', 'State', 'Postal Code', 'Region', 'Product ID', 'Category', 'Sub-Category', 'Product Name', 'Sales', 'Quantity', 'Discount', 'Profit'], dtype='object')
```

```
In [7]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9994 entries, 0 to 9993
Data columns (total 21 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   Row ID                9994 non-null   int64
 1   Order ID              9994 non-null   object
 2   Order Date            9994 non-null   datetime64[ns]
 3   Ship Date             9994 non-null   datetime64[ns]
 4   Ship Mode             9994 non-null   object
 5   Customer ID           9994 non-null   object
 6   Customer Name         9994 non-null   object
 7   Segment              9994 non-null   object
 8   Country               9994 non-null   object
 9   City                 9994 non-null   object
10   State                9994 non-null   object
11   Postal Code           9994 non-null   int64
12   Region               9994 non-null   object
13   Product ID           9994 non-null   object
14   Category              9994 non-null   object
15   Sub-Category          9994 non-null   object
16   Product Name          9994 non-null   object
17   Sales                 9994 non-null   float64
18   Quantity              9994 non-null   int64
19   Discount              9994 non-null   float64
20   Profit               9994 non-null   float64
dtypes: datetime64[ns](2), float64(3), int64(3), object(13)
memory usage: 1.6+ MB
```

```
In [8]: df[['Sales','Discount','Profit','Quantity']].describe()
```

```
Out[8]:
```

	Sales	Discount	Profit	Quantity
count	9994.000000	9994.000000	9994.000000	9994.000000
mean	229.858001	0.156203	28.656896	3.789574
std	623.245101	0.206452	234.260108	2.225110
min	0.444000	0.000000	-6599.978000	1.000000
25%	17.280000	0.000000	1.728750	2.000000
50%	54.490000	0.200000	8.666500	3.000000
75%	209.940000	0.200000	29.364000	5.000000
max	22638.480000	0.800000	8399.976000	14.000000

drop unnecessary columns

```
In [9]: df=df.drop('Row ID',axis=1)
```

```
In [10]: df['Country'].value_counts()
```

```
Out[10]: CountryUnited States    9994Name: count, dtype: int64
```

```
In [11]: df=df.drop('Country',axis=1)
```

```
In [13]: df.drop(['Order ID', 'Ship Date', 'Ship Mode', 'Customer ID'])
```

```
In [14]: df.shape
```

```
Out[14]: (9994, 11)
```

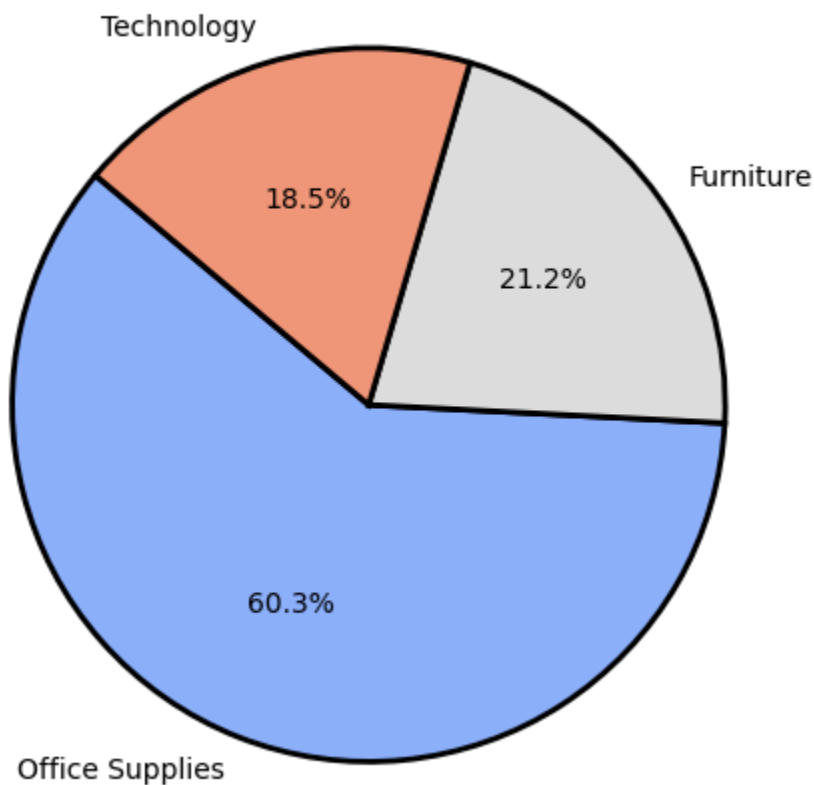
review the product categories available

```
In [15]: category_counts = df['Category'].value_counts()category_counts
```

```
Out[15]: CategoryOffice Supplies    6026Furniture    2121Technology    1847
Name: count, dtype: int64
```

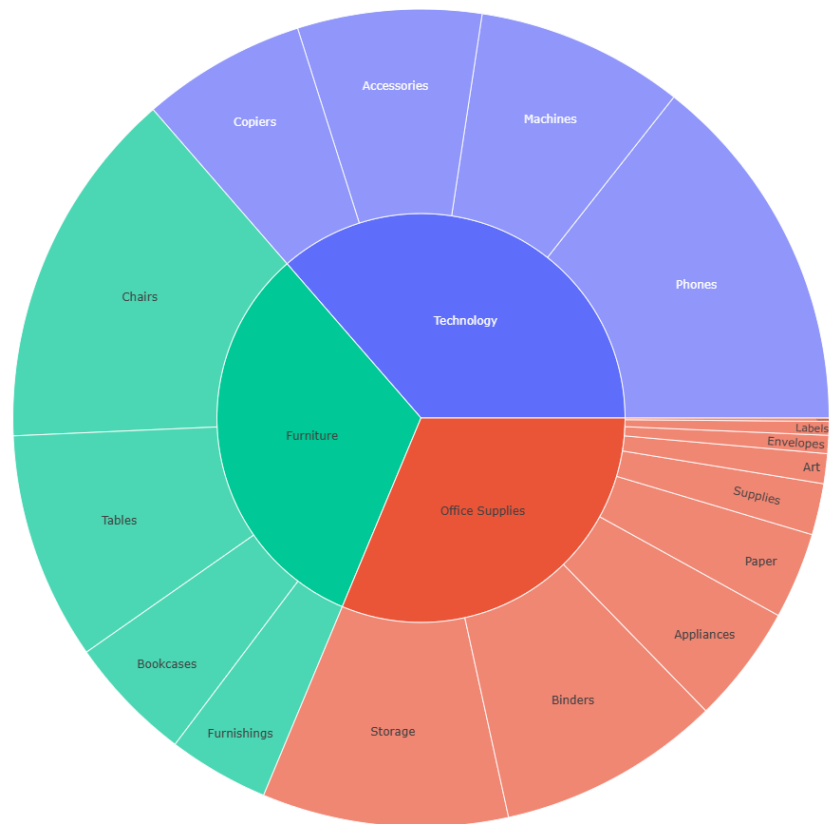
```
In [16]: fig, axs = plt.subplots(figsize=(5, 5))axs.pie(category_counts, labels=category_counts.index)
```

Category Distribution



```
In [41]: import plotly.express as pxfig = px.sunburst(df, path=['Category', 'Sub-Category'])
```

Product Categories & Sub-Categories

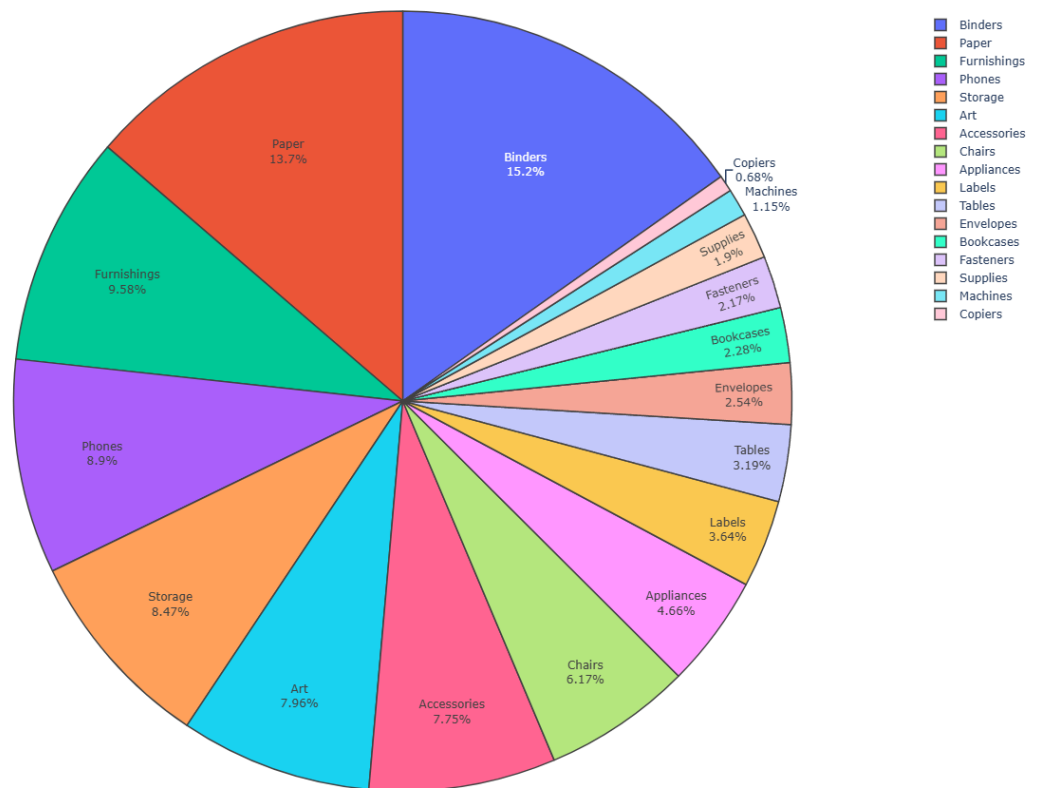


Above graphs indicates->Share of each category & sub-category in Sales->Which sub-category belongs to which category->Share of sub-category for each category(click on any category to get detailed view of each sub-category)

You may get a wrong view of how many products are soldtotal sales in pie plot

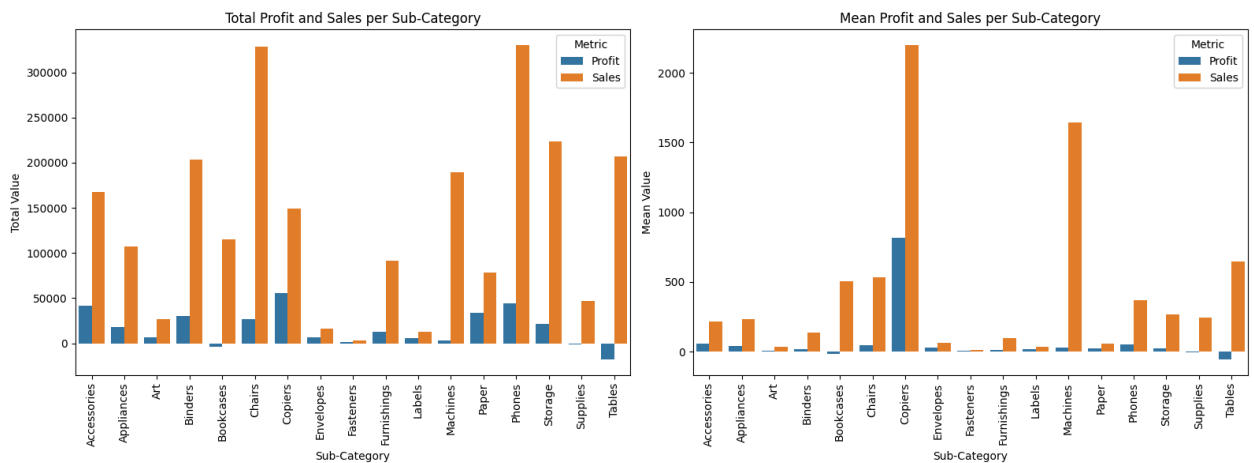
```
In [19]: import plotly.graph_objects as go from plotly.offline import iplotdata=go.Pie(1
```

Pie Plot for Total Sales for each Sub-Category

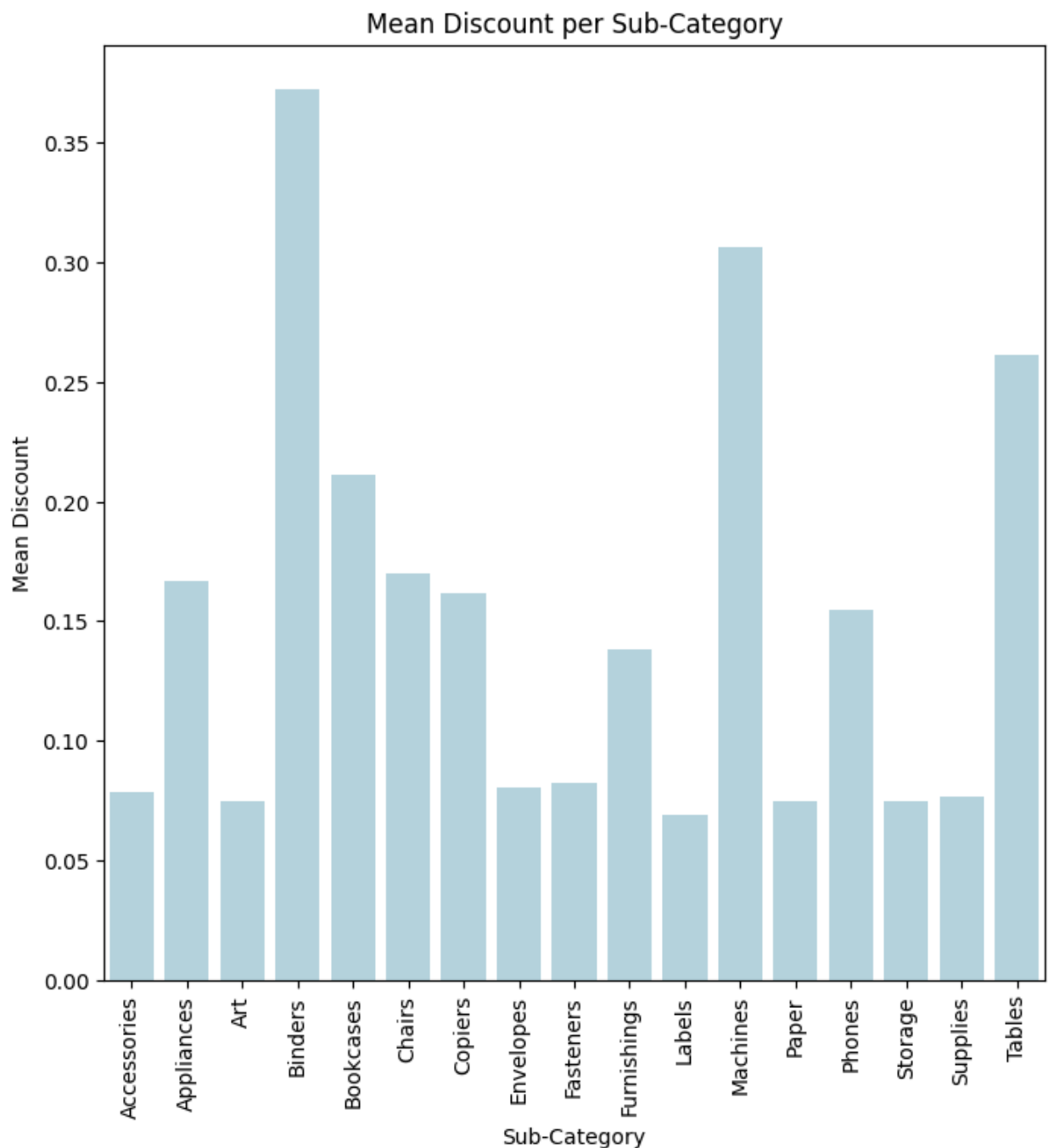


Review of sales and profits

In [20]: `grouped_sum = df.groupby('Sub-Category')[['Profit', 'Sales']].sum().reset_index()`



In [21]: `grouped_discount_mean = df.groupby('Sub-Category')['Discount'].mean().reset_index()`

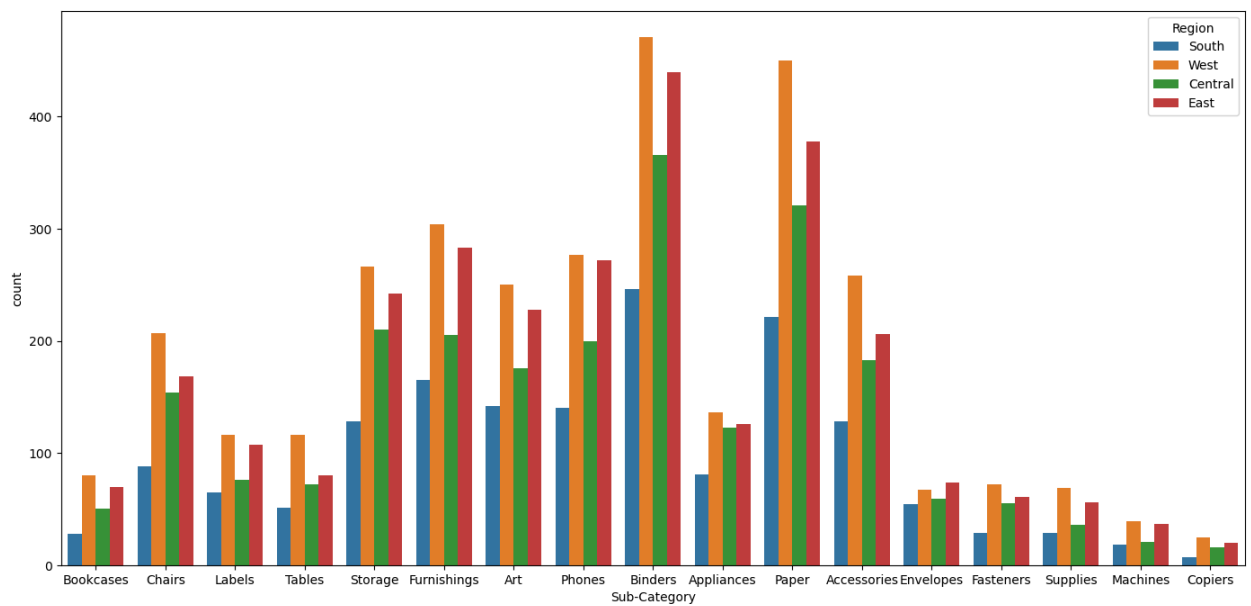


The store's discount system has a problem

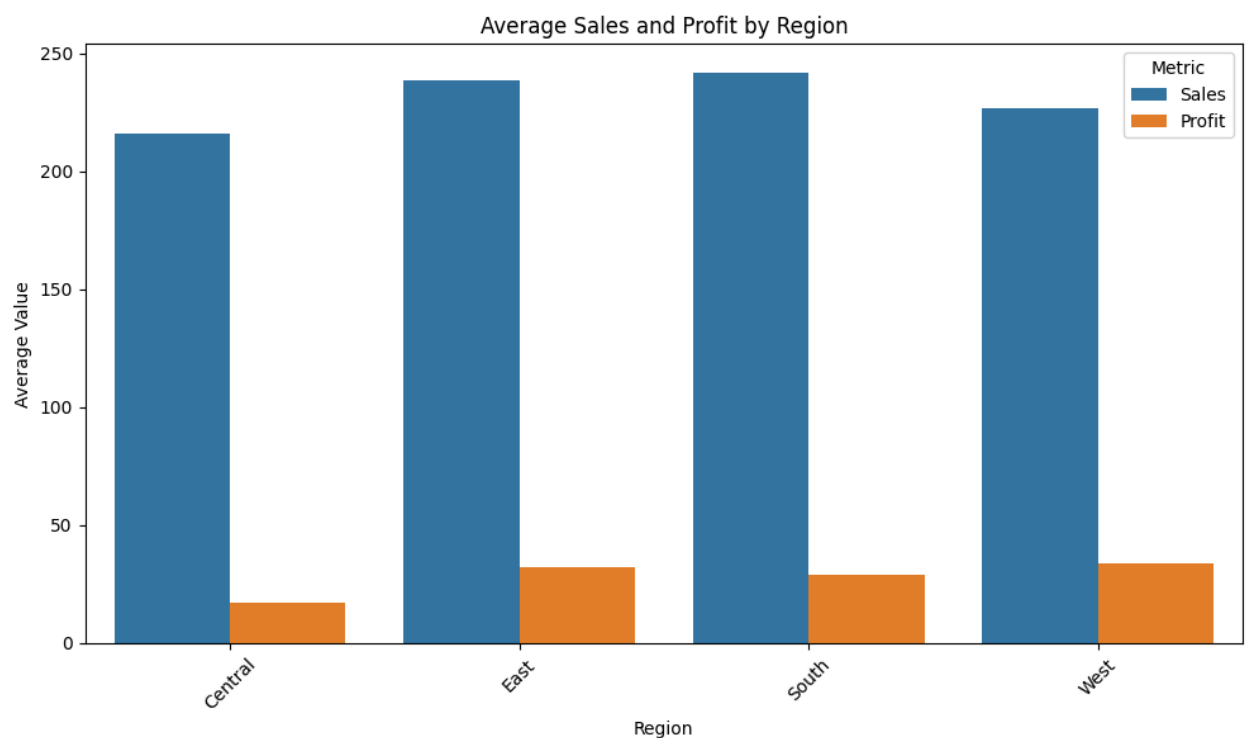
- >Best-selling products can be sold with no discount or with a partial discount-
- >Products that are causing damage to the store should be sold without discount-
- >Products with limited sales and higher profitability should be sold at a higher discount to attract customers

review of the sales of different parts of the US

```
In [22]: plt.figure(figsize=(17,8))sns.countplot(x="Sub-Category", hue="Region", data=c
```



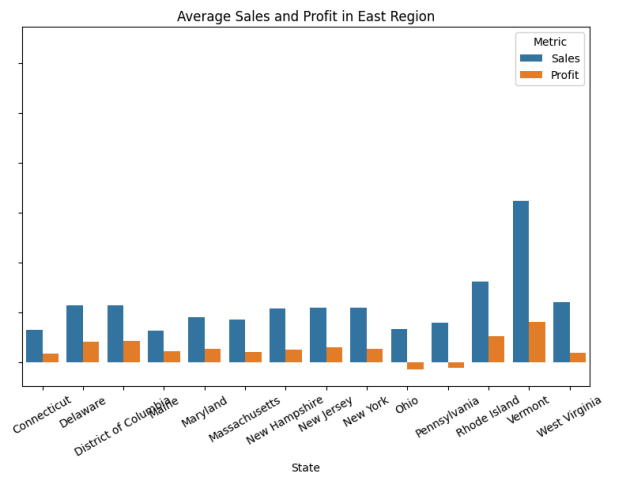
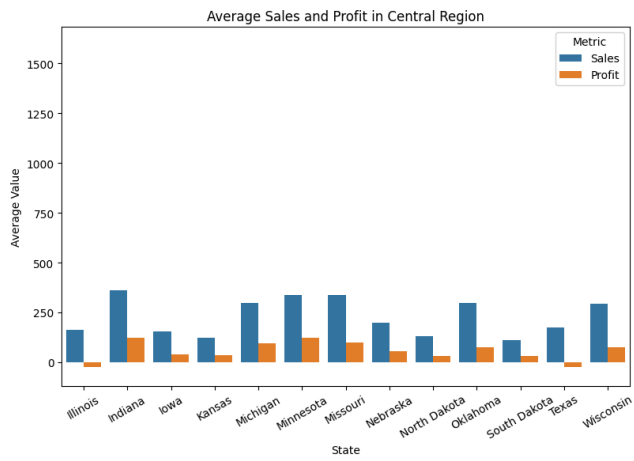
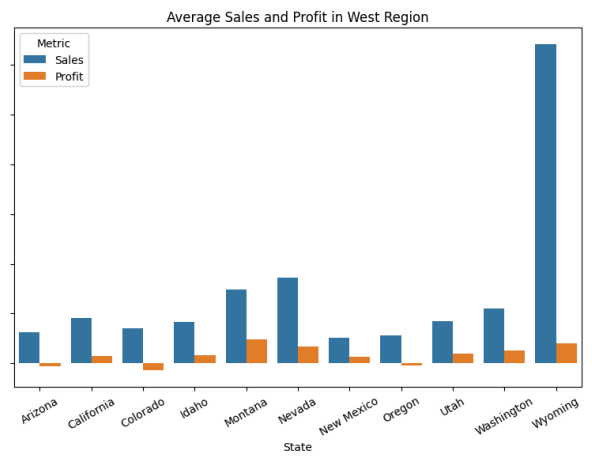
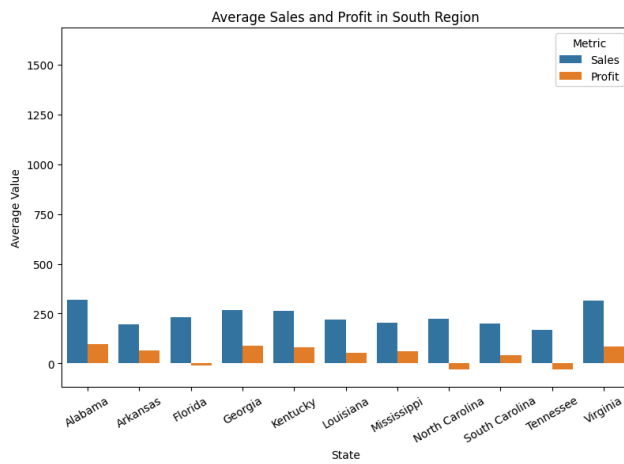
```
In [23]: grouped_data = df.groupby('Region')[['Sales', 'Profit']].mean().reset_index()
```



People in the west and east are more inclined to buy from department superstores than those in the south and center. People in the south have more expensive purchases.

with more details

```
In [24]: regions = df['Region'].unique()fig, axes = plt.subplots(2, 2, figsize=(16, 12))
```



```
In [42]: temp = df[['State', 'City', 'Sales']].groupby(['State', 'City'])['Sales'].sum().r
```


This treemap visualizes the population of the United States, organized by state and then by city. The total population shown is 310,992,341. The size of each rectangle represents the population of that specific city or state. The colors are grouped by state: California (blue), Texas (teal), Florida (orange), Illinois (pink), New York (red), and others. The largest cities, such as New York City (256,368,161) and Los Angeles (175,851,341), are represented by the largest rectangles. The treemap also includes smaller cities and towns, providing a comprehensive view of the population distribution across the country.

State	City	Population
California	Los Angeles	175,851,341
California	San Francisco	112,669,092
California	San Diego	47,521,029
California	San Jose	4,071,229
California	San Antonio	4,113,688
California	San Bernardino	2,008,265
California	San Diego	1,397,138
California	San Francisco	1,113,278
California	San Jose	1,090,114
California	San Bernardino	1,000,000
California	San Diego	900,000
California	San Francisco	800,000
California	San Jose	700,000
California	San Bernardino	600,000
California	San Diego	500,000
California	San Francisco	400,000
California	San Jose	300,000
California	San Bernardino	200,000
California	San Diego	100,000
California	San Francisco	50,000
California	San Jose	25,000
California	San Bernardino	12,500
California	San Diego	6,250
California	San Francisco	3,125
California	San Jose	1,562
California	San Bernardino	781
California	San Diego	390
California	San Francisco	195
California	San Jose	97
California	San Bernardino	48
California	San Diego	24
California	San Francisco	12
California	San Jose	6
California	San Bernardino	3
California	San Diego	1
California	San Francisco	0.5
California	San Jose	0.25
California	San Bernardino	0.125
California	San Diego	0.0625
California	San Francisco	0.03125
California	San Jose	0.015625
California	San Bernardino	0.0078125
California	San Diego	0.00390625
California	San Francisco	0.001953125
California	San Jose	0.0009765625
California	San Bernardino	0.00048828125
California	San Diego	0.000244140625
California	San Francisco	0.0001220703125
California	San Jose	6.103515625e-05
California	San Bernardino	3.0517578125e-05
California	San Diego	1.52587890625e-05
California	San Francisco	7.62939453125e-06
California	San Jose	3.814697265625e-06
California	San Bernardino	1.9073486328125e-06
California	San Diego	9.5367431640625e-07
California	San Francisco	4.76837158203125e-07
California	San Jose	2.384185791015625e-07
California	San Bernardino	1.1920928955078125e-07
California	San Diego	5.9604644775390625e-08
California	San Francisco	2.98023223876953125e-08
California	San Jose	1.4901161193847656e-08
California	San Bernardino	7.450580596923828e-09
California	San Diego	3.725290298461914e-09
California	San Francisco	1.862645149230957e-09
California	San Jose	9.313225746154785e-10
California	San Bernardino	4.656612873077392e-10
California	San Diego	2.328306436538696e-10
California	San Francisco	1.164153218269348e-10
California	San Jose	5.82076609134674e-11
California	San Bernardino	2.91038304567337e-11
California	San Diego	1.455191522836685e-11
California	San Francisco	7.275957614183425e-12
California	San Jose	3.637978807091712e-12
California	San Bernardino	1.818989403545856e-12
California	San Diego	9.09494701772928e-13
California	San Francisco	4.54747350886464e-13
California	San Jose	2.27373675443232e-13
California	San Bernardino	1.13686837721616e-13
California	San Diego	5.6843418860808e-14
California	San Francisco	2.8421709430404e-14
California	San Jose	1.4210854715202e-14
California	San Bernardino	7.105427357601e-15
California	San Diego	3.5527136788005e-15
California	San Francisco	1.77635683940025e-15
California	San Jose	8.88178419700125e-16
California	San Bernardino	4.440892098500625e-16
California	San Diego	2.2204460492503125e-16
California	San Francisco	1.1102230246251562e-16
California	San Jose	5.551115123125781e-17
California	San Bernardino	2.7755575615628905e-17
California	San Diego	1.3877787807814452e-17
California	San Francisco	6.938893903907226e-18
California	San Jose	3.469446951953613e-18
California	San Bernardino	1.7347234759768065e-

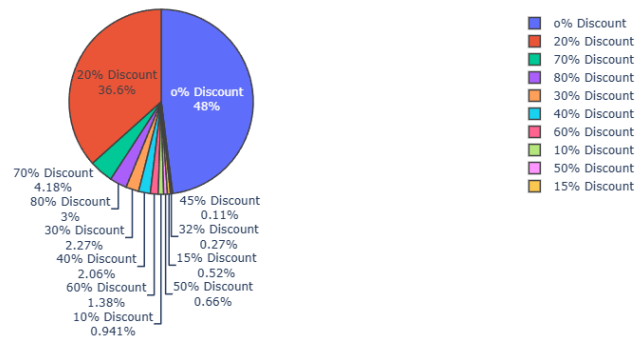
```
df_notime = df[['Segment', 'City', 'State', 'Region', 'Category', 'Sub-Category', 'Sales']]

fig = go.Figure(data=px.scatter(x=df_notime.groupby('State').sum()['Sales'],
```

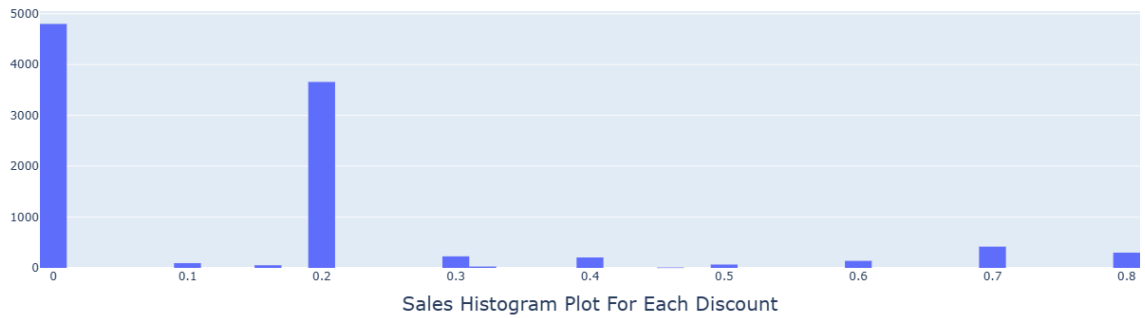
A scatter plot showing the relationship between Sales (X-axis) and Profits (Y-axis). The X-axis ranges from 0 to 450k, and the Y-axis ranges from -20k to 80k. The size of each bubble is proportional to the Profits value. The plot shows a general positive correlation, with a dense cluster of points at low sales and low profits, and a few large bubbles at high sales and high profits.

```
In [35]: labels = ['0% Discount', '20% Discount', '70% Discount', '80% Discount', '30% Disc
```

Discount Pie Plot



```
In [36]: data=go.Histogram(x=df_notime.sort_values('Discount')['Discount'],layout=go.L
```



```
In [38]: from plotly.subplots import make_subplotsfig = make_subplots(6,2,subplot_title
```

Sales Distribution by Discount Levels

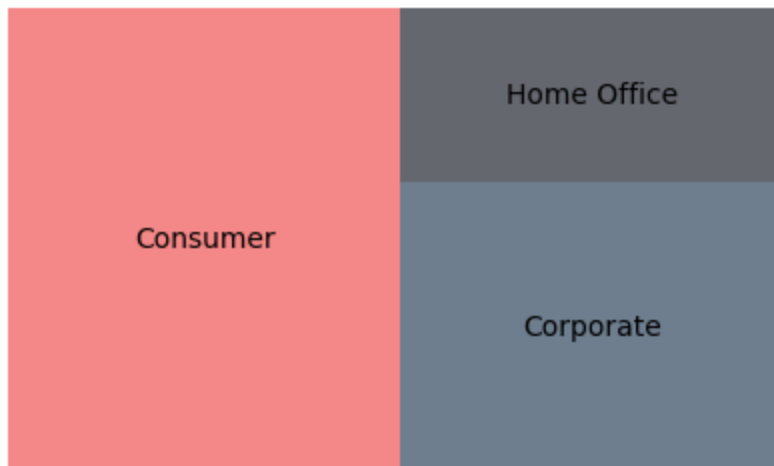


By applying the right policy on discounts, the loss of the store can be avoided in some sections.

Segmentation

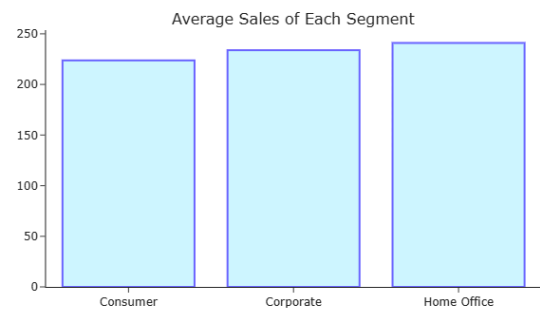
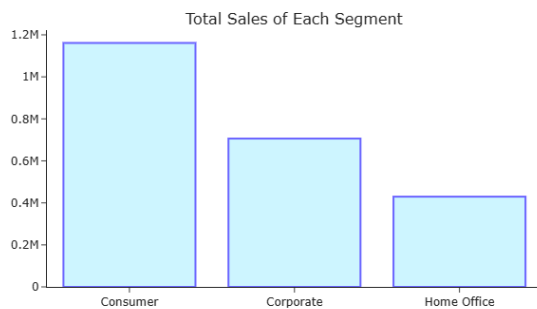
```
In [52]: import squarifydf_tree = df[['Segment', 'Sales']]df_tree = df_tree.groupby(['Se
```

Sales Across Different Segment



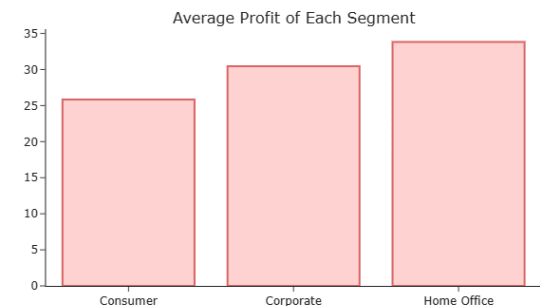
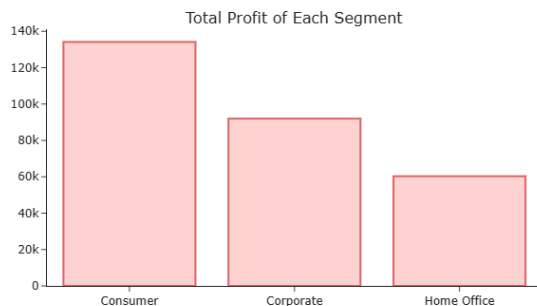
```
In [ ]: segments = df['Segment'].unique()total_sales = [df[df['Segment'] == i]['Sales']
```

Sales Analysis by Segment

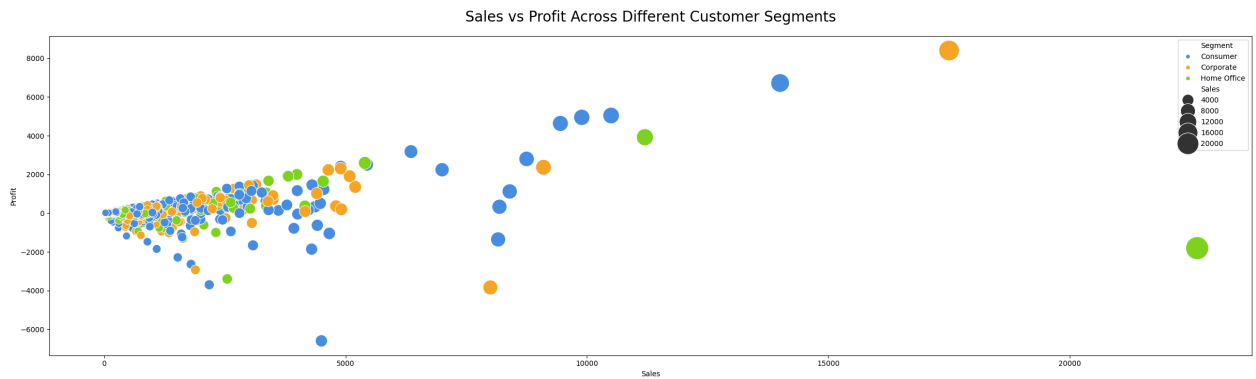


```
In [ ]: total_profit = [df[df['Segment'] == i]['Profit'].sum() for i in segments]avera
```

Profit Analysis by Segment



```
In [71]: df_scatter = df[['Sales', 'Profit', 'Segment']].copy()plt.figure(figsize=[30,8])
```



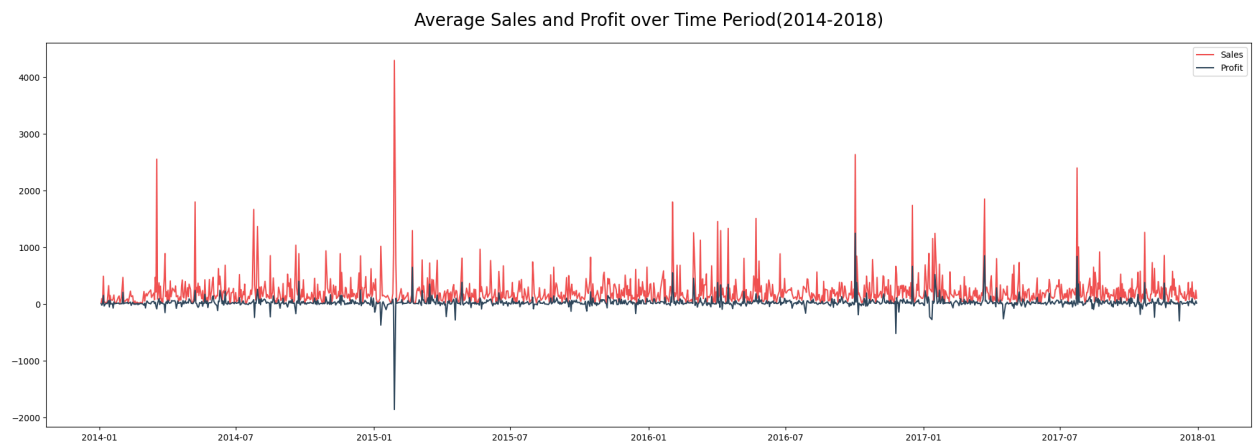
Sales: While the Consumer segment contributes to the highest total sales, the Home Office segment generates higher average sales per transaction. This suggests that consumers are more frequent buyers but tend to make smaller purchases, whereas home office customers, though fewer in number, make larger purchases on average.

Profitability: Similarly, the Consumer segment has the highest total profit, but the Home Office segment achieves greater average profit per transaction. This indicates that the products or services sold to home office customers are more profitable on average.

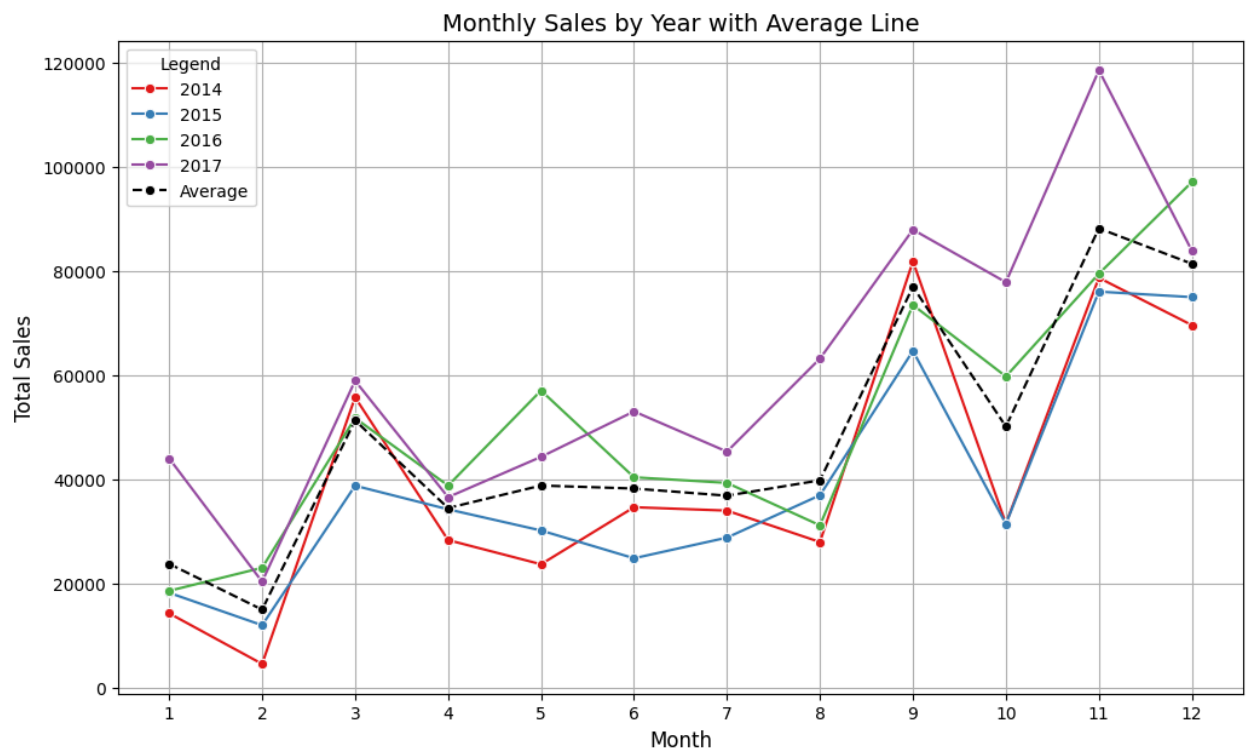
Analyze the Orders & Sales

```
In [58]: df_time = df[['Order Date', 'Segment', 'Sales', 'Quantity', 'Discount', 'Profit']]
```

```
In [59]: df_line = df_time[['Order Date', 'Sales', 'Profit']].sort_values('Order Date')
```



```
In [61]: monthly_sales = df_time.groupby(['Year', 'Month'])['Sales'].sum().reset_index()
```



In [72]: `fig, axes = plt.subplots(3, 1, figsize=(16, 12), sharey=True) axes = axes.flatten()`

